

Prompt:

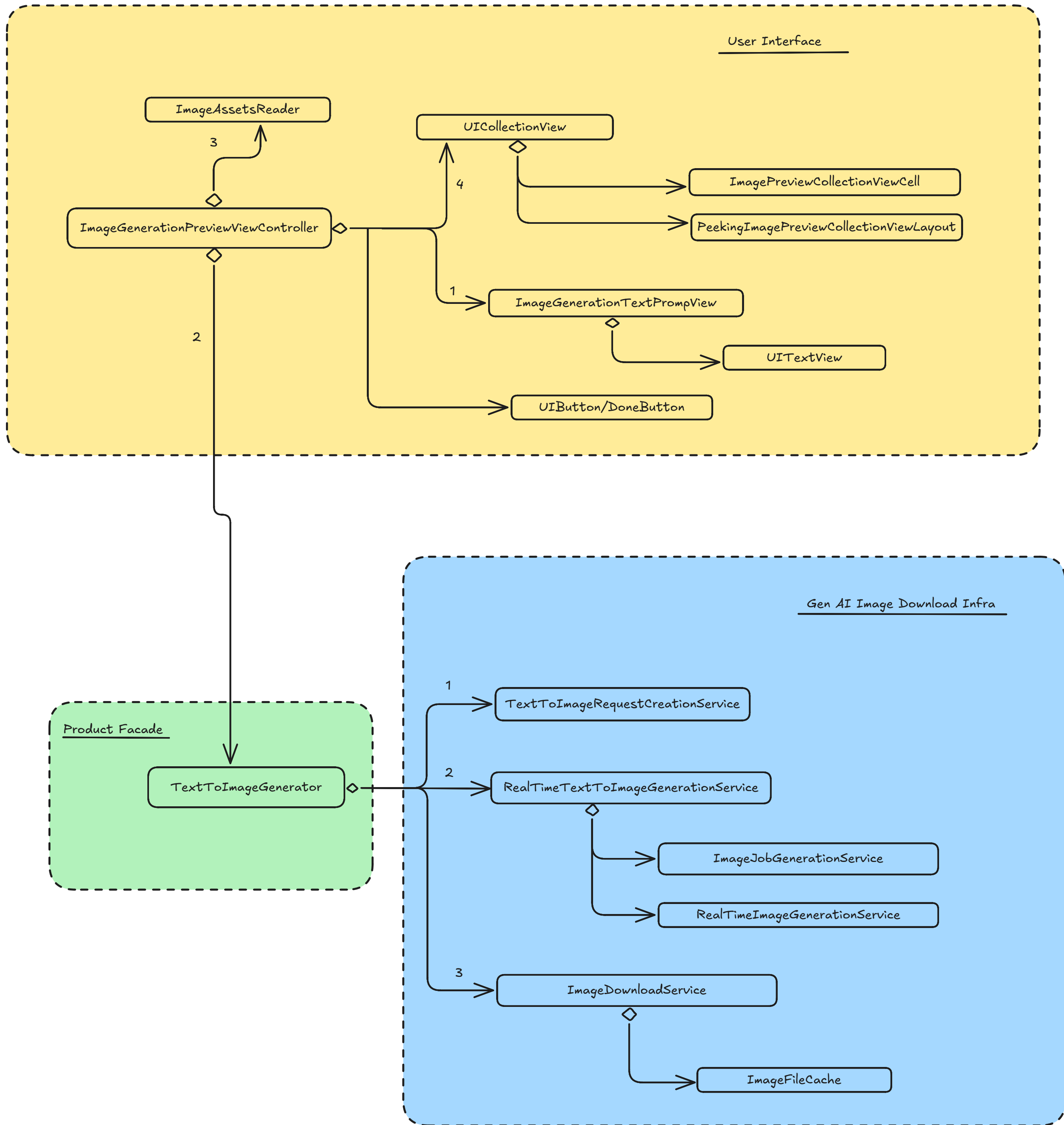
- Design a Gen-AI Image app similar to iOS Image Playground
- The app should generate images from a simple text prompt
- Initially a set of N images should be displayed
- After that the user should be able to swipe to generate more images and we progressively load/generate new images.

Functional Requirements:

- Build out ability to generate images from text input prompts
- User's should have the ability to preview generated images from their input prompts in a clean `UI`
- Should display the first N images generated, then allow users to progressively "load"/"generate" more if they request (on swipe)
- Image component at a bare minimum needs to support:
 - Displaying/rendering downloaded images
 - Displaying a empty state for images loading (and a spinner)
 - A item/selection bar at the bottom
 - A selected/tapped image should go to a full screen preview where one can edit, copy, save. Depending on the context the image playground was opened in this may need to be "customizable." For example in the app go to a full screen preview where u can edit the photo (almost like a post-capture screen), but if within messaging and keyboard tapping the image or done just copies the image over and brings it into the messaging session.

Non-Functional Requirements

- Image Preview component should be smooth and swipeable at a clean 60 fps. Should be able to cleanly render downloaded images and handle and display visual empty state/progress.
- Keep the main thread free 60 fps.
- Image-Gen can be expensive, so we should optimize for latency and apply timeouts here.
- Image Generation should ultimately be a black box for the user and design should be scalable. For a v1 it makes sense to generate from server, but the design should quickly allow to swap the system with an on-device implementation.
- Image Generation (both UI surface and infra) should be designed as a callable and embeddable SDK. It can be displayed in a standalone app, but also displayed and rendered in other experiences for example like the iOS keyboard.



Product Facade API Interface:

[illegible]

Server API Description:

The serverside API is honestly not rocket science for this design so I am going to just describe it at a high level.

1. We will need a HTTP POST endpoint to create an image request on the backend. This is what our `TextToImageRequestCreationService` will call.

- Endpoint: https://image_playground.com/image_generation_request_create?text_prompt=..
- Input: text_prompt which is sent as query param or could be in POST body.
- output:
- Returns a requestID
- Returns text embeddings from text prompt (optional)

2. We will need a HTTP GET endpoint to request generating new images

- Endpoint: `https://image_playground.com/generate_images?requestID=""&count=3`
- Input: simple query params that contain count (num images to generate) and the requestID from step1. If embeddings need to be sent for whatever reason could switch to HTTP POST
- Output:
 - Will return a Job OperationID for SSE in the response payload.
 - Image Generation will not actually begin until a SSE connection is established.

3. We will need a HTTP GET endpoint to "stream" to the client imageAssets as images are generated. This will be managed and called by our RealTimeImageGenerationService. When the server is done generating images it will close the connection. The client will use the jobId returned from step2 and this will kick off image_generation.

- Endpoint: `https://image_playground.com/generate_image_stream?streamID=jobID`
- Input: Literally just a HTTP Get with the JOBID passed in to establish the stream connection
- Output:
 - Will need to handle the raw bytes over the wire sent via `URLSessionDelegate`.
 - As each image is generated literally the server will just return a CDN URL of where to download the image along with some additional metadata.

4. We will need a HTTP POST to delete/clear a image generation request on the backend. Once a client is done with generating images for a text prompt they need to call this to mark it on the backend. The same TextToImageRequestCreationService would call this.

- Endpoint: `https://image_playground.com/image_generation_request_delete?request_id=.`
- Input: Should be self explanatory
- output: Should be self explanatory